

Course Title: **Data Structures and Algorithms** Program: BICTE

Course No. : ICT. Ed. 435

Level: Bachelor

Semester: Third

Nature of course: Theoretical + Practical

Credit Hour: 3 hours (2T+1P)

Teaching Hour: 64 hours (32+32)

---

### 1. Course Description

The purpose of this course is to provide the students with solid foundations in the basic concepts of data structures and algorithms. The main objective of the course is to teach the students how to select and design data structures and algorithms that are appropriate for problems that they might occur. This course is also about showing the correctness of algorithms and studying their computational complexities. This course offers the students a mixture of theoretical knowledge and practical experience. Programming language C will be used for practical work.

### 2. General Objectives

The general objectives of this course are as follows:

- To introduce data abstraction and data representation in memory
- To describe, design and use elementary data structures such as stack, queue, linked list, tree and graph
- To decompose complex programming problems into manageable sub-problems
- To introduce algorithms and their complexity

### 3. Specific Objectives and Contents

| Specific Objectives                                                                                                                                                                                                                                             | Contents                                                                                                                                                                                                                                                                                                                                                                    | LH |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| <ul style="list-style-type: none"><li>• Define Data structure and ADT</li><li>• Define algorithms and its types and asymptotic notations</li></ul>                                                                                                              | <b>Unit 1: Introduction to Data Structures &amp; Algorithms</b><br>1.1 Data type and Abstract data types<br>1.2 Data structures : linear and non-linear, static and dynamic<br>1.3 Algorithms<br>1.3.1 Greedy algorithm<br>1.3.2 Divide and Conquer<br>1.3.3 Backtracking<br>1.3.4 Randomized algorithms<br>1.4 Analysis of Algorithms<br>1.4.1 Big O notation and its rule | 5  |
| <ul style="list-style-type: none"><li>• Demonstrate relationship between array and pointer</li><li>• Implement structure pointers and self-referential pointer</li><li>• Allocate memory dynamically using malloc, calloc, realloc and free functions</li></ul> | <b>Unit 2: Arrays, Pointers and Structures</b><br>2.1 Array and Pointer<br>2.2 Structure and Pointer<br>2.2.1 Structure pointer<br>2.2.2 Self-referential pointer<br>2.3 Dynamic Memory Allocation: malloc, calloc, realloc, free<br><br><b><u>Practical Works</u></b><br>2.1 Write program to illustrate memory allocation dynamically.                                    | 6  |

|                                                                                                                                                                                                                                  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |          |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
| <ul style="list-style-type: none"> <li>Define linked list its type and applications</li> <li>Implement different types of linked list with its operations</li> <li>Make comparison between array list and linked list</li> </ul> | <b>Unit 3: Linked Lists</b><br>3.1 Single Linked list: traversing, searching, inserting and deleting in single linked list<br>3.2 Doubly Linked List: traversing, inserting, creating and deleting in doubly linked list<br>3.3 Circular Linked List: traversing, inserting, creating and deleting in circular linked list<br>3.4 Comparison of Array list and Linked list<br><br><u><b>Practical Works</b></u><br>3.1 Write a program to implement singly, doubly and circular linked list operations | <b>8</b> |
| <ul style="list-style-type: none"> <li>Define and implement stack and stack operations</li> <li>Convert expressions in to different forms: infix, prefix and postfix</li> <li>Describe the applications of the stack</li> </ul>  | <b>Unit 4: Stack</b><br>4.1 Introduction<br>4.2 Array Implementation of Stack<br>4.3 Linked List Implementation of Stack<br>4.4 Applications of Stack<br>4.4.1 Conversion from infix to postfix expression<br>4.4.2 Evaluation of postfix expressions<br><u><b>Practical Works</b></u><br>4.1 Write program to create stack with array and linked list implementation<br>4.2 Write program to illustrate expression conversion and expression evaluation                                               | <b>6</b> |
| <ul style="list-style-type: none"> <li>Define queue and its operations</li> <li>Implement different types of queue</li> </ul>                                                                                                    | <b>Unit 5: Queue</b><br>5.1 Introduction<br>5.2 Array Implementation of Queue<br>5.3 Linked List Implementation of Queue<br>5.4 Circular Queue<br>5.5 Priority Queue.<br><u><b>Practical Works</b></u><br>5.1 Write a program to implement linear, circular and priority queue with array and linked list                                                                                                                                                                                              | <b>6</b> |
| <ul style="list-style-type: none"> <li>Define recursion.</li> <li>Differentiate between recursion and iteration</li> <li>Implement recursion to solve TOH and Fibonacci series</li> </ul>                                        | <b>Unit 6: Recursion</b><br>6.1 Introduction<br>6.2 Examples of Recursion: factorial, fibonacci sequence, Tower of Hanoi(TOH)<br>6.3 Applications and Efficiency of recursion<br><u><b>Practical Works</b></u><br>6.1 Write a program to solve the problem of TOH<br>6.2 Write a program to print Fibonacci series<br>6.3 Write a program to calculate factorial                                                                                                                                       | <b>4</b> |
| <ul style="list-style-type: none"> <li>Define tree and tree operations</li> <li>Create and manipulate</li> </ul>                                                                                                                 | <b>Unit 7: Trees</b><br>7.1 Introduction<br>7.2 Binary Tree : Construction, Traversal (pre-order, in-order,                                                                                                                                                                                                                                                                                                                                                                                            | <b>8</b> |

|                                                                                                                                                                                                                                                                                                                             |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |    |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| Binary tree, BST, AVL tree                                                                                                                                                                                                                                                                                                  | post-order)<br>7.3 Binary Search Tree: Construction, Traversal<br>7.4 AVL tree: Construction, Traversal<br>7.5 Heap: Building a heap<br><u><b>Practical Works</b></u><br>7.1 Write program to implement binary tree.<br>7.2 Write program to implement binary search tree<br>7.3 Write program to implement AVL tree                                                                                                                                                                                                                                                                                                                                                             |    |
| <ul style="list-style-type: none"> <li>Define sorting and its type</li> <li>Demonstrate hashing</li> <li>Illustrate and implement bubble, selection, insertion, merge, quick and heap sort.</li> <li>Implement searching algorithms</li> <li>Identify and compare the efficiency of mentioned sorting algorithms</li> </ul> | <b>Unit 8: Searching, Sorting and Hashing</b><br>8.1 Introduction<br>8.2 Sequential and Binary Search<br>8.3 Hashing: Hash function (truncation, division method, folding, midsquare)<br>8.4 Hash collision and resolution techniques<br>8.5 Sorting Algorithms: Bubble, Selection, Insertion, Merge, Quick and Heap Sort<br>8.6 Efficiency of Sorting Algorithms<br><u><b>Practical Works</b></u><br>8.1 Write program to implement:<br>a) Bubble sort b) Selection sort c) Insertion sort<br>d) Quick sort e) Merge sort f) Heap sort<br>8.2 Write program to implement searching algorithms: binary search and linear search<br>8.3 Write program to implement hash function. | 15 |
| <ul style="list-style-type: none"> <li>Define graph and graph terminologies</li> <li>Explain and implement graph traversal algorithms</li> <li>Find the shortest path using Dijkstra's Algorithm</li> <li>Define MST and implement kruskal's</li> </ul>                                                                     | <b>Unit 9: Graphs</b><br>9.1 Graph Terminology<br>9.2 Directed and undirected graph<br>9.3 Graph Traversal: BFS and DFS<br>9.4 Minimum Spanning Trees: Kruskal Algorithm<br>9.5 Shortest Path Algorithms: Dijkstra's Algorithm<br><u><b>Practical Works</b></u><br>9.1 Write a program to implement graph traversal algorithms : BFS and DFS<br>9.2 Write program to implement Kruskal algorithm and Dijkstra's algorithm                                                                                                                                                                                                                                                        | 6  |

#### 4. Instructional Techniques

The instructional techniques for this course are divided into two groups. First group consists of general instructional techniques applicable to most of the units. The second group consists of specific instructional techniques applicable to particular units.

##### 4.1 General Techniques

Reading materials will be provided to students in each unit. Lecture, Discussion, use of multi-media projector, brain storming, and problem solving methods are used in all units.

## 4.2 Specific Instructional Techniques

Demonstration is an essential instructional technique for all units in this course during teaching learning process. Specifically, demonstration with practical works will be specific instructional technique in this course. The details of suggested instructional techniques are presented below:

| Units                                                | Activities                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Unit 1: Introduction to Data Structures & Algorithms | <ul style="list-style-type: none"><li>• Define and Describe the different types of data structures</li><li>• State different operations occurring in data structures</li><li>• Write a program to implement dynamic memory management functions</li><li>• Explain asymptotic notations and complexity on time and space of algorithm</li><li>• Monitor of students' work by reaching each student and providing feedback for improvement</li><li>• Presentation by students followed by peers' comments and teacher's feedback</li></ul> |
| Unit 3: List                                         | <ul style="list-style-type: none"><li>• Demonstrate operations of linked list with algorithms</li><li>• Lab work in pairs to implement linked list operations</li><li>• Monitor students' work by reaching each student and providing feedback for improvement</li><li>• Presentation by students followed by peers' comments and teacher's feedback</li></ul>                                                                                                                                                                           |
| Unit 2, 4: Array, Pointer and Structure, Stacks      | <ul style="list-style-type: none"><li>• Illustrate array, pointer and structure of C language</li><li>• Illustrate the algorithms of stack operations</li><li>• Lab works in pair to implement stack operations</li><li>• Convert expression in other from one form to another making group and individually</li><li>• Monitoring of students' work by reaching each pair and providing feedback for improvement</li><li>• Presentation by students followed by peers' comments and teacher's feedback</li></ul>                         |
| Unit 5: Queues                                       | <ul style="list-style-type: none"><li>• Demonstrate queue and queue operations with algorithms</li><li>• Lab work in pairs to implement queue operations</li><li>• Group discussion in advantages and limitations of queues</li><li>• Monitoring of students' work by reaching each student and providing feedback for improvement</li><li>• Presentation by students followed by peers' comments and teacher's feedback</li></ul>                                                                                                       |
| Unit 6, 7: Recursion, Trees                          | <ul style="list-style-type: none"><li>• Apply recursive function to calculate factorial, solve TOH problem and generate Fibonacci series</li><li>• Demonstrate operations and types of tree</li><li>• Lab work in pairs to implement BST</li><li>• Trace a working principle of AVL</li><li>• Assign students to create AVL</li></ul>                                                                                                                                                                                                    |

|                                        |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|----------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                        | <ul style="list-style-type: none"> <li>• Monitor students' work by reaching each student and providing feedback for improvement</li> <li>• Presentation by students followed by peers' comments and teacher's feedback</li> </ul>                                                                                                                                                                                                                                                                                                                                                                  |
| Unit 8: Searching, Sorting and Hashing | <ul style="list-style-type: none"> <li>• Demonstrate the working principle of different searching algorithms</li> <li>• Lab work in pair to implement searching algorithms</li> <li>• Implement Hashing</li> <li>• Trace the working principle of different sorting algorithms</li> <li>• Lab work in pair to implement sorting algorithms</li> <li>• Analyze efficiency of sorting algorithms</li> <li>• Monitor students' work by reaching each student and providing feedback for improvement</li> <li>• Presentation by students followed by peers' comments and teacher's feedback</li> </ul> |
| Unit 9: Graphs                         | <ul style="list-style-type: none"> <li>• Explain the graph and graph terminology</li> <li>• Solve the practical problems of shortest path and spanning tree using different algorithms</li> <li>• Assign student to solve graph problems</li> <li>• Lab work in pair to implement graph traversing algorithms</li> <li>• Monitor students' work by reaching each student and providing feedback for improvement</li> <li>• Presentation by students followed by peers' comments and teacher's feedback</li> </ul>                                                                                  |

## 5. Evaluation

Evaluation of students' performance is divided into parts: Internal assessment (theory and practical and internal external examinations (theory and practical)). The distribution of points is given below:

| Internal Assessment Theory | Internal Assessment Practical | Semester Examination (Theoretical exam) | External Practical Exam/Viva | Total Points |
|----------------------------|-------------------------------|-----------------------------------------|------------------------------|--------------|
| 25 Points                  | 15 Points                     | 40 Points                               | 20 Points                    | 100 Points   |

**Note:** Students must pass separately in internal assessment, external practical exam and semester examination.

### 5.1 Internal Assessment (25 Points) of Theoretical Part

Internal assessment will be conducted by subject teacher based on following criteria:

|                                                     |           |
|-----------------------------------------------------|-----------|
| Attendance and learning Activities                  | 5 points  |
| First assignment (Written assignment)               | 5 points  |
| Second assignment (Project work with presentation ) | 10 points |
| Third assignment/written examination                | 5 point   |

|                                                                                                 |           |
|-------------------------------------------------------------------------------------------------|-----------|
| Total                                                                                           | 25 points |
| <b>5.2 Internal Assessment (15 Points) of practical part</b>                                    |           |
| Internal practical assessment will be conducted by subject teacher based on following criteria: |           |
| Attendance and learning Activities                                                              | 5 points  |
| Practical work/project work/lab work                                                            | 10 points |
| Total                                                                                           | 15 points |
| <b>5.3 Semester Final Examination (40 Points) theoretical part</b>                              |           |
| Examination Division, Dean office will conduct final examination at the end of semester.        |           |
| Objective question (Multiple choice questions 10 x 1 point)                                     | 10 Points |
| Subjective questions (6 questions x 5 marks with 'OR' two questions)                            | 30 Points |
| Total                                                                                           | 40 points |

#### 5.4 Practical Exam/Viva (20 Points)

Examination Division, Office of the Dean will appoint an external examiner (ICT teachers working another campus) for conducting practical examination

| Items                                               | Points |
|-----------------------------------------------------|--------|
| Evaluation of Record Book                           | 4      |
| Project work/practical work presentation/skill test | 10     |
| Viva                                                | 6      |
| Total                                               | 20     |

### Recommended Books and References

#### Recommended Books

1. Srivastava, S.K. & Srivastava, D. (2011). *Data structure Through C in Depth*, (2<sup>nd</sup> Ed.), BPB Publication
2. Langsam, Y., Augenstein, M.J. & Tenenbaum, A.M., *Data Structures using C*, Prentice Hall India.